



PREMIUM STUDY GUIDE

MASTER PYTHON IN 30 DAYS

Beginner to Pro — A Complete Day-by-Day Course
for Learners & Teachers

Beginner to Advanced | Syntax · Functions · OOP · Files · APIs · Databases
Includes worked code examples with output, practice exercises, and 2 full
projects

30

DAILY LESSONS

2

FULL PROJECTS

80+

CODE EXAMPLES

30

TEACHER'S NOTES

"Don't worry about writing perfect code — worry about understanding what each line does when it runs."

Table of Contents

WEEK 1 — Foundations (Days 1-7)

Day 1	Introduction to Python & Setup
Day 2	Variables & Data Types
Day 3	Operators
Day 4	Strings & String Methods
Day 5	Lists
Day 6	Tuples & Sets
Day 7	Dictionaries + Week 1 Mini Project

WEEK 2 — Control Flow & Functions (Days 8-14)

Day 8	Conditional Statements: if, elif, else
Day 9	For Loops
Day 10	While Loops, break & continue
Day 11	Functions: def, Parameters, return
Day 12	Function Arguments: Default, *args, **kwargs
Day 13	Lambda Functions & Built-ins
Day 14	Project 1: Contact Book App

WEEK 3 — File Handling, Errors & OOP (Days 15-21)

Day 15	List, Dict & Set Comprehensions
Day 16	String Formatting In Depth
Day 17	File Handling: Reading & Writing Files
Day 18	Exception Handling: try, except, finally
Day 19	OOP Basics: Classes, Objects & Methods
Day 20	OOP: Inheritance & Polymorphism
Day 21	Week 3 Review: Concept Checkpoint

WEEK 4 — Advanced Python & Pro Tools (Days 22-30)

Day 22	Modules, pip & Virtual Environments
Day 23	Decorators
Day 24	Generators & Iterators
Day 25	Regular Expressions
Day 26	Working with JSON & APIs
Day 27	Working with Databases: sqlite3
Day 28	Testing & Debugging: unittest
Day 29	Clean Code & PEP 8 Best Practices
Day 30	Project 2 (Capstone): Expense Tracker

WEEK 1

Python Foundations

Days 1–7: Install Python, learn variables and data types, and master the four core data structures.

Why This Week Matters

Every advanced Python skill — web apps, data analysis, automation — is built entirely out of this week's ideas combined in bigger ways. A shaky grasp of variables, data types, or how lists differ from dictionaries doesn't disappear as the course progresses; it resurfaces as confusing bugs weeks later. Students who feel tempted to rush through Week 1 because it "seems easy" are exactly the ones who benefit most from slowing down here.

This Week's Lessons

DAY 1	Introduction to Python	Setup, print(), comments
DAY 2	Variables & Data Types	int, float, str, bool
DAY 3	Operators	Arithmetic, comparison, logical
DAY 4	Strings & String Methods	Slicing, formatting, methods
DAY 5	Lists	Ordered, changeable collections
DAY 6	Tuples & Sets	Immutable & unique collections
DAY 7	Dictionaries	Key-value pairs + Week 1 project

By the End of This Week, You'll Be Able To

- Install Python and run your first script from the command line
- Explain the difference between int, float, str, and bool, and convert between them
- Use arithmetic, comparison, and logical operators to build an expression
- Store and manipulate collections of data using lists, tuples, sets, and dictionaries
- Choose the right data structure for a given real-world situation

📌 Teacher's Note

If you're teaching this week to a group, resist the urge to move fast just because the material looks simple on the surface. The habits formed in these first seven days — checking a variable's type before using it, knowing instantly which data structure fits which problem — are the same habits that prevent hours of debugging in Week 3 and Week 4. Slow is smooth, and smooth is fast.

Introduction to Python & Setup

📌 Teacher's Note

Most beginners think learning Python means memorizing syntax, but the real skill is learning to think in small, logical steps — Python just happens to be one of the clearest languages for writing those steps down. I always tell new students: don't worry about writing perfect code on Day 1. Worry about understanding what each line does when it runs. Fluency comes from repetition, not from reading.

What Is Python?

Python is a **high-level, general-purpose programming language** known for being readable and beginner-friendly. "High-level" means you write instructions close to plain English rather than dealing with the computer's low-level hardware details. Python is used everywhere: websites (Instagram, Spotify), data science and AI, automation scripts, games, and much more.

Why Python Is a Great First Language

- Its syntax reads almost like English — fewer confusing symbols than many other languages
- It has a massive ecosystem of free libraries for almost any task
- It's used across nearly every industry, from finance to machine learning
- Errors are usually explained in plain, readable messages

Setting Up Python

1. Download Python from **python.org** (choose the latest stable version)
2. During installation on Windows, check **"Add Python to PATH"** — this lets you run Python from any terminal
3. Install a code editor: **VS Code** is the most popular free choice
4. Verify the install by opening a terminal/command prompt and typing: `python --version`

Your First Program

By tradition, every programmer's first program prints "Hello, World!" to the screen. The **print()** function displays text or values on the screen.

CODE

```
print("Hello, World!")
```

OUTPUT

```
Hello, World!
```

Comments — Notes for Humans, Ignored by Python

A comment starts with **#** and is completely ignored when the program runs. Use comments to explain *why* your code does something, not just what it does.

CODE

```
# This program greets the user
print("Hello, World!") # prints text to the screen
```

OUTPUT

```
Hello, World!
```

Running a Python File

Save your code in a file ending in **.py** (e.g. `hello.py`), then run it from the terminal:

CODE

```
python hello.py
```

OUTPUT

```
Hello, World!
```

🔗 **TIP:** VS Code has a built-in "Run" button (a green triangle) once the Python extension is installed — no need to memorize terminal commands right away.

🔗 **PRACTICE:** Install Python and VS Code. Write a program that prints your name and one fact about yourself on two separate lines using two `print()` statements.

Quick Check

What symbol starts a comment in Python?

Answer: The `#` symbol

WEEK 1 — Foundations • DAY 2 OF 30

Variables & Data Types

🔗 Teacher's Note

Variables confuse beginners less than they expect, but data types trip almost everyone up at least once — especially the difference between a number stored as text ("5") and a real number (5). I like to demonstrate this early by trying to add a string and a number and letting the class watch Python raise an error, rather than just describing the rule. A real error message teaches faster than a slide.

What Is a Variable?

A **variable** is a named container that stores a value in memory so you can use it later. In Python, you create a variable simply by assigning a value to a name using `=`.

CODE

```
name = "Aisha"
age = 21
print(name)
print(age)
```

OUTPUT

```
Aisha
21
```

Variable Naming Rules

- Must start with a letter or underscore (not a number)
- Can contain letters, numbers, and underscores — no spaces
- Case-sensitive: `age` and `Age` are different variables
- By convention, Python variables use **snake_case**: `first_name`, not `firstName`

Python's Core Data Types

Type	Name	Example	Description
int	Integer	25	A whole number, positive or negative
float	Floating point	19.99	A number with a decimal point
str	String	"Hello"	Text, wrapped in quotes
bool	Boolean	True / False	One of exactly two values

Checking a Variable's Type

The built-in **`type()`** function tells you what kind of data a variable holds — extremely useful when debugging.

CODE

```
age = 21
price = 19.99
name = "Aisha"
is_student = True

print(type(age))
print(type(price))
print(type(name))
print(type(is_student))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Type Conversion

You can convert between types using **int()**, **float()**, and **str()**. This is essential when combining numbers with text.

CODE

```
age = 21
message = "I am " + str(age) + " years old"
print(message)
```

OUTPUT

```
I am 21 years old
```

⚠ **WATCH OUT:** Trying to combine a number and text directly — like `print("Age: " + 21)` — causes a `TypeError`. Always convert the number to a string first with `str()`.

⇒ **PRACTICE:** Create variables for a product's name (str), price (float), quantity (int), and whether it's in stock (bool). Print all four using a single combined message with `str()` conversion where needed.

Quick Check

What will `type(3.0)` return?

Answer: — because of the decimal point, even though 3.0 looks like a whole number

WEEK 1 — Foundations • DAY 3 OF 30

Operators

📌 Teacher's Note

Operators feel mechanical, but they're the first place students write something that resembles real logic — comparing values, combining conditions. Pay close attention to the difference between `=` (assignment) and `==` (comparison); confusing the two is the single most common early bug I see, and it often produces confusing results rather than a clean error, which makes it harder to spot without deliberate practice.

Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	5 + 3	8
-	Subtraction	10 - 4	6
*	Multiplication	6 * 7	42
/	Division (always returns float)	10 / 4	2.5
//	Floor division (rounds down)	10 // 4	2
%	Modulus (remainder)	10 % 4	2
**	Exponent (power)	2 ** 3	8

CODE

```
print(10 / 4)
print(10 // 4)
print(10 % 4)
print(2 ** 3)
```

OUTPUT

```
2.5
2
2
8
```

Comparison Operators — Return True or False

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	7 > 3	True
<	Less than	7 < 3	False
>=	Greater than or equal to	5 >= 5	True
<=	Less than or equal to	4 <= 3	False

⚠ **WATCH OUT:** A single = assigns a value; a double == compares two values. Writing "if x = 5:" instead of "if x == 5:" is one of the most common beginner mistakes and causes a `SyntaxError` in Python.

Logical Operators — Combine Conditions

Operator	Meaning	Example	Result
and	True only if BOTH sides are True	(5>3) and (2>1)	True
or	True if AT LEAST ONE side is True	(5>3) or (1>2)	True
not	Flips True to False and vice versa	not (5>3)	False

Worked Example: Checking Eligibility

CODE

```
age = 20
has_id = True

is_eligible = (age >= 18) and (has_id == True)
print(is_eligible)
```

OUTPUT

```
True
```

Assignment Operators — Shortcuts

CODE

```
score = 10
score += 5 # same as score = score + 5
print(score)
score *= 2 # same as score = score * 2
print(score)
```

OUTPUT

```
15
30
```

🔗 **PRACTICE:** Create two variables, `hours_worked` and `hourly_rate`. Calculate total pay using `*`, then use a comparison operator to check if total pay is greater than 500.

Quick Check

What does `17 % 5` evaluate to?

Answer: 2 — the remainder after dividing 17 by 5 (3 times, with 2 left over)

WEEK 1 — Foundations • DAY 4 OF 30

Strings & String Methods

🔗 Teacher's Note

Strings are where most real-world Python work actually happens — cleaning text, formatting messages, parsing files — so I spend more time here than the topic seems to deserve on paper. Treat string methods as a toolbox, not a list to memorize: show students the problem first ("this text has extra spaces") and let the method (`.strip()`) feel like a discovered solution rather than an assigned fact.

What Is a String?

A **string** is a sequence of characters wrapped in quotes — either single (`'text'`) or double (`"text"`). Strings are one of the most-used data types in real programs, since almost all user input arrives as text.

String Indexing & Slicing

Each character in a string has a position, starting at **0**. You can extract a single character or a range (a "slice").

CODE

```
word = "Python"
print(word[0])
print(word[5])
print(word[0:3])
print(word[-1])
```

OUTPUT

```
P
n
Pyt
n
```

Index	0	1	2	3	4	5
Character	P	y	t	h	o	n

Negative indexes count from the end: `word[-1]` is always the last character.

Common String Methods

Method	Purpose	Example	Result
<code>.upper()</code>	Converts to uppercase	<code>"hi".upper()</code>	HI
<code>.lower()</code>	Converts to lowercase	<code>"HI".lower()</code>	hi
<code>.strip()</code>	Removes leading/trailing spaces	<code>" hi ".strip()</code>	hi
<code>.replace(a,b)</code>	Replaces text	<code>"cat".replace("c","b")</code>	bat
<code>.split(sep)</code>	Splits into a list	<code>"a,b,c".split(",")</code>	<code>['a', 'b', 'c']</code>
<code>.join(list)</code>	Joins a list into one string	<code>"-".join(["a","b"])</code>	a-b
<code>len(s)</code>	Returns the length	<code>len("hello")</code>	5

f-strings — The Modern Way to Format Text

An f-string lets you embed variables directly inside a string by prefixing it with **f** and wrapping variables in `{}`.

CODE

```
name = "Aisha"
score = 92
print(f"{name} scored {score} percent")
```

OUTPUT

```
Aisha scored 92 percent
```

String Concatenation vs f-strings

CODE

```
name = "Aisha"
# Old way
print("Hello, " + name + "!")
# Modern way (preferred)
print(f"Hello, {name}!")
```

OUTPUT

```
Hello, Aisha!
Hello, Aisha!
```

🔗 **TIP:** Prefer f-strings over + concatenation — they're easier to read, less error-prone, and are the standard used in professional Python code today.

➡ **PRACTICE:** Take the string " Learn Python Today " and use .strip(), .upper(), and .split() to clean it up, capitalize it, then split it into a list of words. Print each step's result.

Quick Check

What does "hello"[1:4] return?

Answer: 'ell' — characters at index 1, 2, and 3 (index 4 is not included)

WEEK 1 — Foundations • DAY 5 OF 30

Lists

📌 Teacher's Note

Lists are the first data structure where order and mutability actually matter, and that's worth naming explicitly. A list can change after you create it — unlike a lot of what students have seen so far — and that mutability is both incredibly useful and a common source of subtle bugs later on, especially once functions start modifying lists that were passed into them.

What Is a List?

A **list** is an ordered, changeable ("mutable") collection of items, written with square brackets `[]`. Lists can hold any data type, even a mix of types.

CODE

```
fruits = ["apple", "banana", "cherry"]
print(fruits)
print(fruits[0])
print(len(fruits))
```

OUTPUT

```
['apple', 'banana', 'cherry']
apple
3
```

Modifying a List

Method	Purpose	Example
.append(x)	Adds an item to the end	fruits.append("mango")
.insert(i,x)	Inserts at a specific position	fruits.insert(1,"kiwi")
.remove(x)	Removes the first matching item	fruits.remove("banana")
.pop(i)	Removes & returns item at index i	fruits.pop(0)
.sort()	Sorts the list in place	fruits.sort()
.reverse()	Reverses the list in place	fruits.reverse()

CODE

```
numbers = [5, 2, 8, 1]
numbers.append(9)
print(numbers)
numbers.sort()
print(numbers)
numbers.pop(0)
print(numbers)
```

OUTPUT

```
[5, 2, 8, 1, 9]
[1, 2, 5, 8, 9]
[2, 5, 8, 9]
```

Looping Through a List

CODE

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

OUTPUT

```
apple
banana
cherry
```

List Slicing

CODE

```
numbers = [10, 20, 30, 40, 50]
print(numbers[1:3])
print(numbers[:2])
print(numbers[2:])
```

OUTPUT

```
[20, 30]
[10, 20]
[30, 40, 50]
```

Checking Membership

CODE

```
fruits = ["apple", "banana", "cherry"]
print("banana" in fruits)
print("mango" in fruits)
```

OUTPUT

```
True
False
```

⚠ **WATCH OUT:** Lists are mutable, meaning changes made inside a function using the same list object will affect the original list everywhere it's referenced — a common source of bugs once functions enter the picture.

🔗 **PRACTICE:** Create a list of 5 favorite movies. Add a new one with `.append()`, remove one with `.remove()`, sort the list alphabetically, then loop through and print each movie with its position number.

Quick Check

What's the difference between `.remove()` and `.pop()`?

Answer: `.remove(x)` deletes by *VALUE*; `.pop(i)` deletes by *INDEX* and also returns the removed item

WEEK 1 — Foundations • DAY 6 OF 30

Tuples & Sets

👨 Teacher's Note

Tuples confuse students mainly because they look like lists but behave differently — the immutability is the entire point, not an arbitrary restriction. I frame the choice this way: use a list when the data might change, use a tuple when it shouldn't. Sets are usually the most fun to teach, because "remove duplicates automatically" is a problem every student has already wished they had a tool for.

Tuples — Ordered but Unchangeable

A **tuple** looks like a list but uses parentheses `()`, and once created, it **cannot be changed** (it's "immutable"). Use a tuple when data shouldn't change during the program — like coordinates or a date.

CODE

```
point = (4, 7)
print(point[0])
print(point[1])
```

OUTPUT

```
4
7
```

CODE

```
point = (4, 7)
point[0] = 10
```

OUTPUT

```
Traceback (most recent call last):
...
TypeError: 'tuple' object does not support item assignment
```

Why Use a Tuple Instead of a List?

Situation	Best choice	Why
A shopping list that will change	List	Items get added/removed
GPS coordinates (latitude, longitude)	Tuple	These two values are fixed together
Days of the week	Tuple	Never changes
A to-do list	List	Tasks get added/completed

Sets — Unordered Collections of Unique Items

A **set** automatically removes duplicate values and has no guaranteed order. Sets are written with curly braces `{}`.

CODE

```
numbers = {1, 2, 2, 3, 3, 3}
print(numbers)
```

OUTPUT

```
{1, 2, 3}
```

Common Set Operations

CODE

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a.union(b))
print(a.intersection(b))
print(a.difference(b))
```

OUTPUT

```
{1, 2, 3, 4}
{2, 3}
{1}
```

Operation	Meaning
union()	All items from both sets, no duplicates
intersection()	Only items that appear in both sets
difference()	Items in the first set but not the second

A Real Use Case: Removing Duplicates

CODE

```
emails = ["a@x.com", "b@x.com", "a@x.com"]
unique_emails = list(set(emails))
print(unique_emails)
```

OUTPUT

```
['a@x.com', 'b@x.com']
```

💡 **TIP:** Converting a list to a set and back is the fastest, most common way to remove duplicates from a list in Python.

⇒ **PRACTICE:** Create a tuple storing your birth (day, month, year). Create two sets of your favorite foods and a friend's favorite foods, then find the overlap using intersection().

Quick Check

Can you change a value inside a tuple after creating it?

Answer: No — tuples are immutable. You would need to create a new tuple instead.

WEEK 1 — Foundations • DAY 7 OF 30

Dictionaries + Week 1 Mini Project

👨‍🏫 Teacher's Note

Dictionaries are, in my experience, the single most under-appreciated data structure for beginners — once students see that real-world information (a person, a product, a student record) naturally has named fields rather than just an ordered list, dictionaries stop feeling abstract and start feeling like the obvious choice. This is a good day to explicitly connect the four data structures learned this week and ask: which one would you use for which real situation?

What Is a Dictionary?

A **dictionary** stores data as **key-value pairs** instead of a simple ordered list — perfect for representing real-world things that have named properties, like a person or a product.

CODE

```
student = {"name": "Aisha", "age": 21, "grade": "A"}
print(student["name"])
print(student["age"])
```

OUTPUT

```
Aisha
21
```

Modifying a Dictionary

CODE

```
student = {"name": "Aisha", "age": 21}
student["age"] = 22      # update a value
student["grade"] = "A"   # add a new key
print(student)
del student["grade"]     # remove a key
print(student)
```

OUTPUT

```
{'name': 'Aisha', 'age': 22, 'grade': 'A'}
{'name': 'Aisha', 'age': 22}
```

Useful Dictionary Methods

Method	Purpose	Example
.keys()	All the keys	student.keys()
.values()	All the values	student.values()
.items()	All key-value pairs	student.items()
.get(key)	Safely gets a value (no error if missing)	student.get("age")

Looping Through a Dictionary

CODE

```
student = {"name": "Aisha", "age": 21, "grade": "A"}
for key, value in student.items():
    print(f"{key}: {value}")
```

OUTPUT

```
name: Aisha
age: 21
grade: A
```

Avoiding Errors with .get()

CODE

```
student = {"name": "Aisha"}
print(student["age"])
```

OUTPUT

```
Traceback (most recent call last):
...
KeyError: 'age'
```

CODE

```
student = {"name": "Aisha"}
print(student.get("age", "Not specified"))
```

OUTPUT

Not specified

⚠ **WATCH OUT:** Accessing a missing key with square brackets (`student["age"]`) crashes the program with a `KeyError`. Use `.get(key, default)` whenever the key might not exist.

Choosing the Right Data Structure

Need to...	Use
Keep items in order and allow changes	List
Store a fixed collection that shouldn't change	Tuple
Store unique items and don't care about order	Set
Store named fields describing one "thing"	Dictionary

⇒ **PRACTICE:** Create a dictionary representing a product with keys: name, price, and in_stock. Print a formatted sentence describing it using an f-string, then safely check for a "discount" key using `.get()` with a default of 0.

Quick Check

Which method lets you access a dictionary value without risking a `KeyError`?

Answer: `.get(key, default_value)`

Week 1 Mini Project: Contact Card Generator

Combine everything from this week into a small program that stores and displays a contact.

CODE

```
contact = {
    "name": "Ravi Kumar",
    "phone": "9876543210",
    "emails": ["ravi@example.com", "ravi.k@work.com"],
    "is_favorite": True
}

print(f"Name: {contact['name']}")
print(f"Phone: {contact['phone']}")
print("Emails:")
for email in contact["emails"]:
    print(f"  - {email}")
print(f"Favorite contact? {contact['is_favorite']}")
```

OUTPUT

```
Name: Ravi Kumar
Phone: 9876543210
Emails:
  - ravi@example.com
  - ravi.k@work.com
Favorite contact? True
```

⇒ **PRACTICE:** Extend the Contact Card program: add a list of 3 contacts (a list of dictionaries) and loop through all of them, printing each one's details.

WEEK 2

Control Flow & Functions

Days 8–14: Make decisions with if/else, repeat work with loops, and package logic into reusable functions.

Why This Week Matters

This is the week Python stops being a calculator and starts being a real program. if/else teaches it to make a choice; loops teach it to repeat work without you copy-pasting code; functions teach it to package logic into a reusable, nameable unit. Individually these feel like small tricks, but together they are the core structure behind almost every real program ever written, in any language.

This Week's Lessons

DAY 8	Conditional Statements	if, elif, else
DAY 9	For Loops	Repeating over a sequence
DAY 10	While Loops	break, continue
DAY 11	Functions	def, parameters, return
DAY 12	Function Arguments	Default, *args, **kwargs
DAY 13	Lambda & Built-ins	map, filter, sorted
DAY 14	Project 1	Contact Book command-line app

By the End of This Week, You'll Be Able To

- Write a program that branches based on one or more conditions
- Choose correctly between a for loop and a while loop for a given task
- Write your own function with parameters and a return value
- Use default arguments and *args/**kwargs to make a function more flexible
- Combine everything into a small, working command-line application

📌 Teacher's Note

Watch for the moment a student starts combining two of this week's tools without being told to — for example, writing a function that contains a loop with an if statement inside it. That's the real signal the underlying structure has clicked, not just the individual keywords. I treat that moment as more meaningful than any quiz score.

Conditional Statements: if, elif, else

👨‍🏫 Teacher's Note

If/elif/else is the first time a program actually makes a decision instead of just executing top to bottom, and that shift deserves to be named out loud. I ask students to write out their logic in plain English first — "if it's raining AND I have an umbrella, go outside" — before touching Python syntax. Translating logic into code is a different skill from having the logic in the first place, and separating the two steps dramatically reduces frustration.

Why Programs Need Decisions

Real programs rarely do the exact same thing every time — they react to input, data, or circumstances. The **if** statement is how Python makes a decision: it runs a block of code only when a condition is True.

Basic if Statement

CODE

```
age = 20
if age >= 18:
    print("You are an adult")
```

OUTPUT

```
You are an adult
```

⚠️ **WATCH OUT:** Python uses INDENTATION (spaces) to define which code belongs inside the if block — not curly braces like many other languages. A consistent 4-space indent is the standard convention.

if / else

CODE

```
age = 15
if age >= 18:
    print("You are an adult")
else:
    print("You are a minor")
```

OUTPUT

```
You are a minor
```

if / elif / else — Multiple Conditions

CODE

```
score = 75
if score >= 90:
    print("Grade: A")
elif score >= 75:
    print("Grade: B")
elif score >= 50:
    print("Grade: C")
else:
    print("Grade: F")
```

OUTPUT

```
Grade: B
```

Python checks each condition top to bottom and stops at the **first** one that's True — the remaining elif/else blocks are skipped entirely.

Combining Conditions

CODE

```
age = 20
has_ticket = True
if age >= 18 and has_ticket:
    print("Entry allowed")
else:
    print("Entry denied")
```

OUTPUT

Entry allowed

Nested if Statements

CODE

```
weather = "sunny"
temperature = 35
if weather == "sunny":
    if temperature > 30:
        print("It's a hot sunny day")
    else:
        print("Pleasant sunny day")
```

OUTPUT

It's a hot sunny day

💡 **TIP:** If you find yourself nesting if statements more than two levels deep, it's often a sign the logic can be simplified using 'and'/'or' instead — deeply nested code is harder to read and debug.

⇒ **PRACTICE:** Write a program that checks a person's age and prints "Child" (under 13), "Teenager" (13-19), or "Adult" (20+) using if/elif/else.

Quick Check

What happens if none of the if/elif conditions are True and there's no else block?

Answer: Nothing runs — Python simply skips the entire if/elif chain and moves to the next line of code.

WEEK 2 — Control Flow • DAY 9 OF 30

For Loops

📌 Teacher's Note

The for loop is where Python starts to feel powerful rather than just correct — a five-line loop can process a thousand items as easily as five. The concept that trips people up isn't the loop itself, it's the loop variable: many beginners don't initially realize that "item" in "for item in list" is just a name they chose, not a special keyword. A quick renaming exercise fixes this misunderstanding fast.

What Is a Loop?

A **loop** repeats a block of code multiple times. The **for** loop repeats once for each item in a sequence (a list, string, or range of numbers) — use it whenever you know what you're iterating over.

Looping Through a List

CODE

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

OUTPUT

apple
banana
cherry

The range() Function

range() generates a sequence of numbers — extremely useful for repeating something a specific number of times.

CODE

```
for i in range(5):  
    print(i)
```

OUTPUT

```
0  
1  
2  
3  
4
```

Call	Produces
range(5)	0, 1, 2, 3, 4 (stops before 5)
range(2, 6)	2, 3, 4, 5
range(0, 10, 2)	0, 2, 4, 6, 8 (step of 2)

Looping Through a String

CODE

```
for letter in "Python":  
    print(letter)
```

OUTPUT

```
P  
Y  
t  
h  
o  
n
```

Using enumerate() for Index + Value

When you need both the position and the value while looping, **enumerate()** is cleaner than manually tracking a counter.

CODE

```
fruits = ["apple", "banana", "cherry"]  
for index, fruit in enumerate(fruits):  
    print(index, fruit)
```

OUTPUT

```
0 apple  
1 banana  
2 cherry
```

A Practical Example: Summing a List

CODE

```
numbers = [10, 20, 30, 40]  
total = 0  
for num in numbers:  
    total += num  
print(f"Total: {total}")
```

OUTPUT

```
Total: 100
```

💡 **TIP:** Whenever you're processing every item in a list, string, or range one by one, reach for a for loop first — it's the most common and most readable choice in Python.

🔗 **PRACTICE:** Write a program that loops through numbers 1 to 20 and prints only the even numbers, using `range()` and an `if` statement inside the loop.

Quick Check

What does `range(1, 10, 3)` produce?

Answer: 1, 4, 7 — starting at 1, stepping by 3, stopping before 10

WEEK 2 — Control Flow • DAY 10 OF 30

While Loops, `break` & `continue`

🔗 Teacher's Note

While loops are more dangerous than `for` loops in one specific way: it's entirely possible to write one that never ends. I make a point of showing an infinite loop on purpose (and how to stop it, `Ctrl+C`) so the fear of that mistake becomes familiar rather than mysterious. Understanding `break` and `continue` is really about learning to interrupt a loop's default behavior on purpose, which is a small taste of the control real programs need.

The while Loop

A **while** loop repeats as long as a condition stays `True`. Use it when you don't know in advance exactly how many times you'll need to repeat — for example, repeating until a user enters valid input.

CODE

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

OUTPUT

```
1
2
3
4
5
```

⚠️ **WATCH OUT:** If the condition in a while loop never becomes `False`, the loop runs forever (an 'infinite loop'). Always make sure something inside the loop eventually changes the condition — like `count += 1` above.

`break` — Exit a Loop Early

break immediately stops the loop, even if the condition is still `True`.

CODE

```
count = 1
while True:
    print(count)
    if count == 3:
        break
    count += 1
```

OUTPUT

```
1
2
3
```

`continue` — Skip to the Next Iteration

continue skips the rest of the current loop cycle and jumps straight to the next one.

CODE

```
for num in range(1, 6):  
    if num == 3:  
        continue  
    print(num)
```

OUTPUT

```
1  
2  
4  
5
```

A Practical Example: Input Validation Loop

This is one of the most common real uses of while — repeating until the user provides valid input.

CODE

```
password = ""  
while password != "python123":  
    password = input("Enter the password: ")  
print("Access granted!")
```

OUTPUT

```
Enter the password: wrong  
Enter the password: python123  
Access granted!
```

for vs while — Which One to Use

Situation	Best choice
Looping over a known list, string, or range	for loop
Repeating until a condition changes (e.g. user input)	while loop
Repeating exactly N times	for loop with range(N)
Running until the user chooses to quit	while loop

⇒ **PRACTICE:** Write a while loop that keeps asking the user to guess a secret number (stored as a variable) and prints "Correct!" and stops once they guess it. Use break to exit as soon as the guess is right.

Quick Check

What's the difference between break and continue?

Answer: break exits the loop completely; continue skips only the current iteration and moves to the next one

WEEK 2 — Control Flow • DAY 11 OF 30

Functions: def, Parameters, return

👨🏻‍🏫 Teacher's Note

Functions are the first big abstraction in this course — the idea that you can name a piece of logic once and reuse it without rewriting it. The concept of a 'return value' versus code that just prints something is subtle and worth over-explaining: a function that prints a result and a function that returns a result look similar on the surface but behave completely differently once you try to use the result elsewhere in a program.

Why Use Functions?

A **function** is a named, reusable block of code. Instead of copy-pasting the same logic everywhere, you write it once and call it by name whenever you need it — making programs shorter, easier to read, and easier to fix (change the logic in one place, not ten).

Defining and Calling a Function

CODE

```
def greet():  
    print("Hello there!")  
  
greet()  
greet()
```

OUTPUT

```
Hello there!  
Hello there!
```

Functions with Parameters

A **parameter** lets you pass information into a function so it can behave differently each time it's called.

CODE

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Aisha")  
greet("Ravi")
```

OUTPUT

```
Hello, Aisha!  
Hello, Ravi!
```

The return Statement

return sends a value back out of the function so it can be stored or used elsewhere — this is very different from just printing inside the function.

CODE

```
def add(a, b):  
    return a + b  
  
result = add(5, 3)  
print(result)  
print(add(10, 20) + 1)
```

OUTPUT

```
8  
31
```

⚠ **WATCH OUT:** A function that only print()s a result cannot have that result reused elsewhere — the printed text disappears. Use return whenever the caller needs to use the result in further calculations.

Multiple Parameters and Return Values

CODE

```
def describe_student(name, score):  
    if score >= 50:  
        status = "Pass"  
    else:  
        status = "Fail"  
    return f"{name}: {status}"  
  
print(describe_student("Ravi", 65))  
print(describe_student("Meera", 40))
```

OUTPUT

```
Ravi: Pass  
Meera: Fail
```

Why Functions Reduce Bugs

If the same calculation (say, tax computation) appears in five places and the tax rate changes, you must fix it in five places — and it's easy to miss one. Wrapped in a single function, you fix it in exactly one place.

► **PRACTICE:** Write a function `calculate_area(length, width)` that returns the area of a rectangle. Call it with three different sets of values and print each result.

Quick Check

What's the difference between a parameter and an argument?

Answer: A parameter is the named placeholder in the function definition; an argument is the actual value you pass in when calling the function.

WEEK 2 — Control Flow • DAY 12 OF 30

Function Arguments: Default, *args, **kwargs

📌 Teacher's Note

Default arguments and `*args/**kwargs` feel like advanced syntax, but conceptually they're just about flexibility: letting a function handle a range of situations instead of exactly one. I introduce these only after students have felt the pain of writing three nearly-identical functions for slightly different inputs — the tool lands much better once the problem it solves has already been personally experienced.

Default Arguments

A default argument gives a parameter a fallback value, used only if the caller doesn't provide one.

CODE

```
def greet(name, greeting="Hello"):
    print(f'{greeting}, {name}!')

greet("Aisha")
greet("Ravi", "Welcome")
```

OUTPUT

```
Hello, Aisha!
Welcome, Ravi!
```

Keyword Arguments

You can pass arguments by name instead of by position — this makes function calls with many parameters much clearer.

CODE

```
def describe_pet(name, animal, age):
    print(f'{name} is a {age}-year-old {animal}!')

describe_pet(name="Rex", age=3, animal="dog")
```

OUTPUT

```
Rex is a 3-year-old dog
```

*args — Accepting Any Number of Positional Arguments

***args** lets a function accept any number of extra arguments, collected into a tuple.

CODE

```
def total(*numbers):
    return sum(numbers)

print(total(1, 2, 3))
print(total(10, 20, 30, 40))
```

OUTPUT

```
6
100
```

kwargs — Accepting Any Number of Named Arguments

kwargs collects extra named arguments into a dictionary — useful when you don't know in advance what fields will be provided.

CODE

```
def print_profile(**details):
    for key, value in details.items():
        print(f"{key}: {value}")

print_profile(name="Aisha", age=21, city="Delhi")
```

OUTPUT

```
name: Aisha
age: 21
city: Delhi
```

When to Use Each

Tool	Use when...
Default argument	A parameter usually has one common value but occasionally needs to change
Keyword argument	A function has many parameters and you want the call to be self-explanatory
*args	You don't know how many positional values will be passed in
**kwargs	You don't know how many named values will be passed in

 **TIP:** You rarely need *args and **kwargs as a beginner, but recognizing them is important — they appear constantly in real-world Python libraries.

⇒ **PRACTICE:** Write a function `order_summary(item, quantity=1, **extras)` that prints the item, quantity, and any extra details passed in as keyword arguments (like `gift_wrap=True`).

Quick Check

What data type does *args collect its values into?

Answer: A tuple

WEEK 2 — Control Flow • DAY 13 OF 30

Lambda Functions & Built-ins: map, filter, sorted

Teacher's Note

Lambda functions are simultaneously overused and under-explained. I tell students plainly: a lambda is just a function without a name, useful for small throwaway logic, not a replacement for regular functions. Pairing it with `map()` and `filter()` is a good moment to reinforce that Python treats functions as values you can pass around — a concept that will resurface later with decorators.

What Is a Lambda Function?

A **lambda** is a small, anonymous (unnamed) function written in a single line — useful for short, throwaway logic that doesn't deserve a full function definition.

CODE

```
square = lambda x: x * x
print(square(5))
```

OUTPUT

```
25
```

Compare to a regular function doing the same thing:

CODE

```
def square(x):  
    return x * x  
print(square(5))
```

OUTPUT

```
25
```

⚠ **WATCH OUT:** Lambda functions are best for small, simple expressions. If your logic needs more than one line or an if/else block, write a regular function with `def` instead — it will be far more readable.

map() — Apply a Function to Every Item

CODE

```
numbers = [1, 2, 3, 4]  
squared = list(map(lambda x: x * x, numbers))  
print(squared)
```

OUTPUT

```
[1, 4, 9, 16]
```

filter() — Keep Only Items That Match a Condition

CODE

```
numbers = [1, 2, 3, 4, 5, 6]  
evens = list(filter(lambda x: x % 2 == 0, numbers))  
print(evens)
```

OUTPUT

```
[2, 4, 6]
```

sorted() with a Custom Key

The **key** parameter lets you tell `sorted()` exactly what to sort by — extremely useful for sorting lists of dictionaries or tuples.

CODE

```
students = [("Ravi", 82), ("Aisha", 95), ("Meera", 70)]  
by_score = sorted(students, key=lambda s: s[1], reverse=True)  
print(by_score)
```

OUTPUT

```
[('Aisha', 95), ('Ravi', 82), ('Meera', 70)]
```

Other Handy Built-in Functions

Function	Purpose	Example
<code>sum()</code>	Adds all items in a list	<code>sum([1,2,3]) → 6</code>
<code>max()</code> / <code>min()</code>	Largest / smallest item	<code>max([4,9,2]) → 9</code>
<code>len()</code>	Number of items	<code>len([1,2,3]) → 3</code>
<code>sorted()</code>	Returns a new sorted list	<code>sorted([3,1,2]) → [1,2,3]</code>
<code>any()</code> / <code>all()</code>	True if any/all items are True	<code>any([False,True]) → True</code>

⇒ **PRACTICE:** Given a list of prices [250, 80, 999, 45, 600], use `filter()` with a lambda to find all prices above 100, then use `sorted()` to sort the result from highest to lowest.

Quick Check

What does `list(map(lambda x: x + 1, [1,2,3]))` return?

Answer: [2, 3, 4] — 1 is added to every item

Project 1: Contact Book App

Project Overview

You're going to build a **command-line Contact Book** — a program that lets a user add, view, search, and delete contacts, all stored in memory using a list of dictionaries. This project uses **every skill from Week 1 and Week 2**: variables, lists, dictionaries, loops, conditionals, and functions.

Planning Before Coding

Before writing a single line, sketch out what the program needs to do:

- Store multiple contacts, each with a name and phone number
- Let the user add a new contact
- Let the user view all contacts
- Let the user search for a contact by name
- Let the user delete a contact
- Keep asking until the user chooses to quit

Step 1: Data Storage

CODE

```
contacts = [] # will hold dictionaries like {"name": "...", "phone": "..."}
```

Step 2: The Core Functions

CODE

```
def add_contact(name, phone):
    contacts.append({"name": name, "phone": phone})
    print(f"Added {name}.")

def view_contacts():
    if not contacts:
        print("No contacts saved yet.")
        return
    for i, c in enumerate(contacts, start=1):
        print(f"{i}. {c['name']} - {c['phone']}")

def search_contact(name):
    for c in contacts:
        if c["name"].lower() == name.lower():
            print(f"Found: {c['name']} - {c['phone']}")
            return
    print("Contact not found.")

def delete_contact(name):
    for c in contacts:
        if c["name"].lower() == name.lower():
            contacts.remove(c)
            print(f"Deleted {name}.")
            return
    print("Contact not found.")
```

Step 3: The Menu Loop

CODE

```
def main():
    while True:
        print("\n--- Contact Book ---")
        print("1. Add contact")
        print("2. View all contacts")
        print("3. Search contact")
        print("4. Delete contact")
        print("5. Quit")
        choice = input("Choose an option: ")

        if choice == "1":
            name = input("Name: ")
            phone = input("Phone: ")
            add_contact(name, phone)
        elif choice == "2":
            view_contacts()
        elif choice == "3":
            name = input("Name to search: ")
            search_contact(name)
        elif choice == "4":
            name = input("Name to delete: ")
            delete_contact(name)
        elif choice == "5":
            print("Goodbye!")
            break
        else:
            print("Invalid choice, try again.")

main()
```

Sample Run

Here's what a real session looks like once you run the full program in your terminal:

TERMINAL

```
--- Contact Book ---
1. Add contact
2. View all contacts
3. Search contact
4. Delete contact
5. Quit
Choose an option: 1
Name: Ravi
Phone: 9876543210
Added Ravi.

--- Contact Book ---
Choose an option: 2
1. Ravi - 9876543210

--- Contact Book ---
Choose an option: 5
Goodbye!
```

 **TIP:** This kind of menu-driven while-True loop with numbered options is one of the most common patterns in beginner command-line programs — you'll reuse this exact structure constantly.

Extend the Project (Recommended)

- ☐ Prevent duplicate names when adding a contact
- ☐ Add an "edit contact" option using the same search pattern
- ☐ Save contacts to a file so they aren't lost when the program closes (preview of Week 3's file handling lesson!)
- ☐ Sort contacts alphabetically before displaying them

⇒ **PRACTICE:** Build the Contact Book exactly as shown, test all 5 menu options, then add at least one extension from the list above on your own.

WEEK 3

File Handling, Errors & OOP

Days 15–21: Write more compact code with comprehensions, work with real files, handle errors gracefully, and learn Object-Oriented Programming.

Why This Week Matters

This week marks a shift from writing small scripts to writing more structured, resilient programs. File handling means your program's data can outlive the program itself. Exception handling means a program can survive an unexpected situation instead of crashing. And classes — the biggest single idea in this course — let you model real-world things as objects that carry both data and behavior together.

This Week's Lessons

DAY 15	Comprehensions	Compact list/dict building
DAY 16	String Formatting	f-strings in depth
DAY 17	File Handling	Reading and writing files
DAY 18	Exception Handling	try, except, finally
DAY 19	OOP Basics	Classes, objects, methods
DAY 20	OOP: Inheritance	Reuse and extend classes
DAY 21	Week 3 Review	Concept checkpoint

By the End of This Week, You'll Be Able To

- Rewrite a for-loop as a compact comprehension when appropriate
- Read from and write to a text file safely using 'with open()'
- Anticipate and handle errors using try/except instead of letting a program crash
- Define your own class with attributes and methods
- Use inheritance to reuse behavior across related classes

📌 Teacher's Note

This week tends to be the most fun to teach because the payoff is immediate — a program that used to crash on bad input now handles it gracefully, and a jumble of related variables becomes one clean object. Use that energy. Classes in particular deserve patience: don't rush from 'here's a class' to 'here's inheritance' in the same session, even though this guide covers both back to back for reference.

List, Dict & Set Comprehensions

👨‍🏫 Teacher's Note

Comprehensions are notorious for looking intimidating before they click and trivial afterward. I always teach the equivalent for-loop version first, then show the comprehension as a compressed rewrite of exactly the same logic — never introduce the compressed syntax before the longhand version has been fully understood, or it reads as magic instead of as a shortcut.

Why Comprehensions?

A **list comprehension** is a compact way to build a list from an existing sequence, combining a loop and (optionally) a condition into a single line. It's not a new concept — it's a shorter way to write a pattern you already know.

The Long Way: A Regular Loop

CODE

```
squares = []
for x in range(1, 6):
    squares.append(x * x)
print(squares)
```

OUTPUT

```
[1, 4, 9, 16, 25]
```

The Short Way: A List Comprehension

CODE

```
squares = [x * x for x in range(1, 6)]
print(squares)
```

OUTPUT

```
[1, 4, 9, 16, 25]
```

The pattern is: `[expression for item in sequence]` — read it as "expression, for each item in this sequence."

Adding a Condition

CODE

```
evens = [x for x in range(1, 11) if x % 2 == 0]
print(evens)
```

OUTPUT

```
[2, 4, 6, 8, 10]
```

Compare to the loop version — the comprehension says the exact same thing in one line:

CODE

```
evens = []
for x in range(1, 11):
    if x % 2 == 0:
        evens.append(x)
print(evens)
```

OUTPUT

```
[2, 4, 6, 8, 10]
```

Dictionary Comprehensions

CODE

```
names = ["Aisha", "Ravi", "Meera"]
name_lengths = {name: len(name) for name in names}
print(name_lengths)
```

OUTPUT

```
{'Aisha': 5, 'Ravi': 4, 'Meera': 5}
```

Set Comprehensions

CODE

```
words = ["cat", "dog", "cat", "bird"]
unique_lengths = {len(w) for w in words}
print(unique_lengths)
```

OUTPUT

```
{3, 4}
```

⚠ **WATCH OUT:** Comprehensions are meant to make code SHORTER and CLEARER — if a comprehension needs more than one condition or nested loop to the point it's hard to read, switch back to a regular for loop instead.

⇒ **PRACTICE:** Given a list of numbers 1-30, write a list comprehension that keeps only numbers divisible by 3, then a dictionary comprehension that maps each of those numbers to its square.

Quick Check

What does `[x for x in range(5)]` produce?

Answer: `[0, 1, 2, 3, 4]` — a plain list built directly from `range(5)`

WEEK 3 — Structure & OOP • DAY 16 OF 30

String Formatting In Depth: f-strings

📌 Teacher's Note

F-strings feel like a minor convenience, but they represent a real shift in how comfortable students become with combining text and data — and readable output is an underrated professional skill. A program that prints a confusing wall of text is much harder to debug than one that prints a clearly labeled result, and f-strings make that labeling nearly effortless.

f-strings in Depth

You met f-strings in Week 1 — today, a closer look at what they can really do, since they're the standard way professional Python code formats text.

Formatting Numbers

CODE

```
price = 1234.5678
print(f"Price: {price:.2f}")
print(f"Price: {price:,.2f}")
```

OUTPUT

```
Price: 1234.57
Price: 1,234.57
```

Format spec	Meaning	Example
:.2f	2 decimal places	3.14159 → 3.14
;	Thousands separator	1000000 → 1,000,000
[:,.2f]	Both combined	1234567.891 → 1,234,567.89
:>10	Right-align in 10 spaces	for building simple tables
:05d	Pad with zeros to 5 digits	42 → 00042

Expressions Inside f-strings

You can put any valid Python expression inside the curly braces, not just a plain variable.

CODE

```
a = 5
b = 3
print(f"{a} + {b} = {a + b}")
print(f"Is a bigger? {a > b}")
```

OUTPUT

```
5 + 3 = 8
Is a bigger? True
```

Calling Functions Inside f-strings

CODE

```
name = "aisha sharma"
print(f"Name: {name.title()}")
```

OUTPUT

```
Name: Aisha Sharma
```

Multi-line f-strings for Reports

CODE

```
name = "Ravi"
score = 87
print(f"""
Report Card
-----
Student: {name}
Score:    {score}
Grade:    {'Pass' if score >= 50 else 'Fail'}
""")
```

OUTPUT

```
Report Card
-----
Student: Ravi
Score:    87
Grade:    Pass
```

💡 **TIP:** The pattern `value_if_true if condition else value_if_false` used inside an f-string above is called a 'conditional expression' — a compact one-line if/else that returns a value instead of running a full block.

⇒ **PRACTICE:** Given a product name, price (a float), and quantity, print a neatly formatted receipt line using f-string number formatting, showing the total cost to 2 decimal places with a comma thousands separator.

Quick Check

What does `f'{7/3:.2f}'` print?

Answer: 2.33 — the division result rounded to 2 decimal places

File Handling: Reading & Writing Files

👨‍🏫 Teacher's Note

File handling is the first time a Python program has an effect outside of its own memory — data that exists after the program stops running. That persistence is a genuinely new idea worth pausing on. It's also where the 'with open(...) as f' pattern deserves real explanation rather than being handed over as boilerplate to memorize: it exists specifically to guarantee a file gets closed properly, even if something goes wrong.

Why File Handling Matters

Every program you've written so far loses all its data the moment it stops running. File handling lets a program **save data permanently** — and read it back later, even after restarting.

Opening a File — The Safe Way

Always use **with open(...) as f:** — it automatically closes the file when you're done, even if an error happens partway through.

CODE

```
with open("notes.txt", "w") as f:
    f.write("Hello, this is my first file!")
```

Mode	Meaning
"r"	Read (default) — file must already exist
"w"	Write — creates the file, OVERWRITES if it already exists
"a"	Append — adds to the end without erasing existing content
"r+"	Read and write

Reading a Whole File

CODE

```
with open("notes.txt", "r") as f:
    content = f.read()
print(content)
```

OUTPUT

```
Hello, this is my first file!
```

Reading Line by Line

CODE

```
with open("notes.txt", "w") as f:
    f.write("Apple\nBanana\nCherry")

with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())
```

OUTPUT

```
Apple
Banana
Cherry
```

🔗 **TIP:** Use `.strip()` when printing lines read from a file — each line includes a trailing newline character (`\n`) that would otherwise create extra blank lines in your output.

Appending to a File

CODE

```
with open("notes.txt", "a") as f:
    f.write("\nDate added\n")
```

Working with CSV-style Data

A very common real-world pattern: saving simple records as comma-separated lines.

CODE

```
students = [("Aisha", 92), ("Ravi", 78)]
with open("scores.csv", "w") as f:
    for name, score in students:
        f.write(f"{name},{score}\n")

with open("scores.csv", "r") as f:
    for line in f:
        name, score = line.strip().split(",")
        print(f"{name} scored {score}")
```

OUTPUT

```
Aisha scored 92
Ravi scored 78
```

⚠ **WATCH OUT:** Opening a file that doesn't exist in 'r' mode raises a `FileNotFoundError`. Either check the file exists first, or (better) wrap the code in a `try/except` — covered next lesson.

⇒ **PRACTICE:** Write a program that asks the user for 3 favorite movies (using `input()` in a loop) and saves them to a file called `movies.txt`, one per line. Then read the file back and print each movie with a number in front of it.

Quick Check

Which file mode should you use to add new data without erasing what's already there?

Answer: "a" (append mode)

WEEK 3 — Structure & OOP • DAY 18 OF 30

Exception Handling: try, except, finally

👨 Teacher's Note

Exception handling is where I explicitly teach the difference between a bug and an error a program should expect. Not every unusual situation is a mistake in the code — a user might type text instead of a number, a file might not exist — and `try/except` is how a program stays running gracefully instead of crashing. I ask students to list what could realistically go wrong with a piece of code before wrapping it in a `try` block, rather than adding `except` clauses reflexively.

What Is an Exception?

An **exception** is an error that occurs while a program is running, causing it to stop immediately — unless you handle it. Common examples: dividing by zero, accessing a missing dictionary key, or converting invalid text to a number.

CODE

```
number = int("abc")
```

OUTPUT

```
Traceback (most recent call last):
...
ValueError: invalid literal for int() with base 10: 'abc'
```

try / except — Handling Errors Gracefully

CODE

```
try:
    number = int(input("Enter a number: "))
    print(f"You entered {number}")
except ValueError:
    print("That wasn't a valid number.")
```

OUTPUT

```
Enter a number: abc
That wasn't a valid number.
```

Catching Specific Exception Types

Exception	When it happens
ValueError	Wrong type of value, e.g. int("abc")
ZeroDivisionError	Dividing by zero
KeyError	Accessing a missing dictionary key
IndexError	Accessing a list index that doesn't exist
FileNotFoundError	Trying to open a file that doesn't exist
TypeError	Combining incompatible types, e.g. "5" + 5

CODE

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
```

OUTPUT

```
Cannot divide by zero!
```

Handling Multiple Exception Types

CODE

```
try:
    numbers = [1, 2, 3]
    print(numbers[5])
except IndexError:
    print("That index doesn't exist.")
except Exception as e:
    print(f"Something else went wrong: {e}")
```

OUTPUT

```
That index doesn't exist.
```

finally — Always Runs

Code inside **finally** runs no matter what — whether an error occurred or not. It's commonly used for cleanup, like closing a resource.

CODE

```
try:
    print(10 / 2)
finally:
    print("This always runs")
```

OUTPUT

```
5.0
This always runs
```

🔗 **TIP:** Wrap only the specific lines that might fail inside try — putting your ENTIRE program inside one giant try block makes it much harder to tell what actually went wrong.

⚠️ **WATCH OUT:** A bare 'except:' with no error type catches EVERYTHING, including mistakes you didn't anticipate, which can hide real bugs. Always catch specific exception types when you can.

⇒ **PRACTICE:** Write a program that asks the user for two numbers and divides them, using try/except to handle both ValueError (invalid input) and ZeroDivisionError (dividing by zero) with a friendly message for each.

Quick Check

What's the difference between except ValueError and a bare except:?

Answer: except ValueError only catches that specific error type; a bare except: catches every possible error, which can hide unexpected bugs.

WEEK 3 — Structure & OOP • DAY 19 OF 30

OOP Basics: Classes, Objects & Methods

👨‍🏫 Teacher's Note

Classes are the single biggest conceptual leap in this entire course, and I don't rush it. The key reframing is this: a class isn't just a container for data, it's a blueprint for creating many similar objects that each carry their own copy of that data. I spend real time showing two different objects made from the same class, each with different attribute values, before writing a single method — the students need to see 'separate objects' before 'shared behavior' makes sense.

What Is Object-Oriented Programming?

Object-Oriented Programming (OOP) organizes code around **objects** — bundles of data (attributes) and behavior (methods) modeled after real-world things. A **class** is the blueprint; an **object** is one specific thing built from that blueprint.

Defining a Class

CODE

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        print(f"{self.name} says Woof!")
```

`__init__` is the *constructor* — it runs automatically when you create a new object, setting up its initial data. **self** refers to the specific object being created or used.

Creating Objects (Instances)

CODE

```
dog1 = Dog("Rex", "Labrador")
dog2 = Dog("Bella", "Poodle")

print(dog1.name)
print(dog2.breed)
dog1.bark()
dog2.bark()
```

OUTPUT

```
Rex
Poodle
Rex says Woof!
Bella says Woof!
```

Notice: **dog1** and **dog2** are two completely separate objects, each with its own **name** and **breed** — even though they were built from the exact same class.

Attributes vs Methods

Term	Meaning	Example
Attribute	A piece of data stored on an object	dog1.name
Method	A function defined inside a class	dog1.bark()
self	Refers to the current object inside a method	self.name

A More Complete Example

CODE

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited {amount}. New balance: {self.balance}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds.")
        else:
            self.balance -= amount
            print(f"Withdrew {amount}. New balance: {self.balance}")

account = BankAccount("Aisha", 1000)
account.deposit(500)
account.withdraw(2000)
account.withdraw(300)
```

OUTPUT

```
Deposited 500. New balance: 1500
Insufficient funds.
Withdrew 300. New balance: 1200
```

⚠ **WATCH OUT:** Forgetting 'self' as the first parameter of a method is one of the most common beginner mistakes with classes — Python needs self to know WHICH object's data to use.

🔗 **PRACTICE:** Create a class Book with attributes title, author, and pages. Add a method summary() that prints a formatted description. Create two different Book objects and call summary() on each.

Quick Check

What is the difference between a class and an object?

Answer: A class is the blueprint/template; an object is one specific instance created from that blueprint, with its own data.

WEEK 3 — Structure & OOP • DAY 20 OF 30

OOP: Inheritance & Polymorphism

📌 Teacher's Note

Inheritance is best taught with an example students can picture instantly — animals, vehicles, shapes — rather than an abstract one. The point I emphasize is reuse: inheritance exists so you don't have to rewrite shared behavior for every related class. Polymorphism tends to click only after inheritance is solid, so I don't introduce the two on the same day if I can help it, even though this guide covers both together for reference.

What Is Inheritance?

Inheritance lets one class (the *child*) reuse and extend the attributes and methods of another class (the *parent*), avoiding duplicate code between closely related classes.

A Basic Example

CODE

```
class Animal:
    def __init__(self, name):
        self.name = name

    def eat(self):
        print(f"{self.name} is eating.")

class Dog(Animal):
    def bark(self):
        print(f"{self.name} says Woof!")

rex = Dog("Rex")
rex.eat()
rex.bark()
```

OUTPUT

```
Rex is eating.
Rex says Woof!
```

Dog automatically gets the **eat()** method and the **__init__** constructor from **Animal**, without rewriting either — it only needed to add what's new (**bark()**).

Overriding a Method

A child class can redefine a parent method to give it different behavior — this is called **overriding**.

CODE

```
class Animal:
    def speak(self):
        print("Some generic animal sound")

class Cat(Animal):
    def speak(self):
        print("Meow!")

Animal().speak()
Cat().speak()
```

OUTPUT

```
Some generic animal sound
Meow!
```

Using super() to Extend the Parent's Behavior

CODE

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)    # runs Animal's __init__ first
        self.breed = breed

d = Dog("Rex", "Labrador")
print(d.name, d.breed)
```

OUTPUT

```
Rex Labrador
```

super() lets the child class call the parent's version of a method — commonly used inside **__init__** to avoid re-writing the parent's setup logic.

Polymorphism — Same Method Name, Different Behavior

Polymorphism means different classes can be used interchangeably if they share a method name, even though each implements it differently.

CODE

```
animals = [Cat(), Dog("Rex", "Lab")]
for a in animals:
    if hasattr(a, "speak"):
        a.speak()
```

OUTPUT

Meow!

Why Inheritance Matters

Without inheritance	With inheritance
Repeat shared attributes/methods in every related class	Define once in a parent class
A bug fix must be repeated in every class	Fix once in the parent — every child benefits
Hard to see the relationship between classes	Class hierarchy makes relationships explicit

⇒ **PRACTICE:** Create a parent class `Vehicle` with attributes `make` and `model` and a method `describe()`. Create two child classes `Car` and `Motorcycle` that each add one unique attribute and override `describe()` to include it.

Quick Check

What does `super().__init__()` do inside a child class's constructor?

Answer: It calls the parent class's `__init__` method, running the parent's setup logic before the child adds its own.

WEEK 3 — Structure & OOP • DAY 21 OF 30

Week 3 Review: Concept Checkpoint

👩 **Teacher's Note**

By the end of Week 3, students have every core building block of the language. What I watch for in Project 2's checkpoint isn't whether the code is elegant — it's whether a student can decide, unprompted, which structure fits which problem: a list here, a dictionary there, a class when the data needs both fields and behavior. That decision-making is the real skill this week was building toward.

Week 3 Recap

Day	Skill
15	List, dict, and set comprehensions
16	f-strings: number formatting, expressions
17	Reading and writing files with <code>with open()</code>
18	<code>try/except/finally</code> for graceful error handling
19	Classes, objects, <code>__init__</code> , <code>self</code>
20	Inheritance, method overriding, <code>super()</code> , polymorphism

Concept Checkpoint: Put It All Together

This short exercise combines every Week 3 topic into one program — a great gauge of what's solid and what needs review before Week 4.

CODE

```
class Student:
    def __init__(self, name, scores):
        self.name = name
        self.scores = scores

    def average(self):
        return sum(self.scores) / len(self.scores) if self.scores else 0

    def save_report(self):
        try:
            with open(f"{self.name}_report.txt", "w") as f:
                f.write(f"Student: {self.name}\n")
                f.write(f"Average: {self.average():.2f}\n")
            print("Report saved.")
        except Exception as e:
            print(f"Could not save report: {e}")

students = [Student("Aisha", [88, 92, 79]), Student("Ravi", [60, 55, 70])]
top_students = [s.name for s in students if s.average() >= 75]
print(f"Students averaging 75+: {top_students}")

for s in students:
    s.save_report()
```

OUTPUT

```
Students averaging 75+: ['Aisha']
Report saved.
Report saved.
```

Notice how many Week 3 ideas appear in this one short program: a **class** with a method, a **list comprehension** to filter students, an **f-string** with number formatting, and **try/except** around the file-writing code.

🔗 **TIP:** If any single piece of that program felt unfamiliar, that's exactly the day worth revisiting before moving into Week 4 — the advanced topics ahead build directly on all five of these ideas.

⇒ **PRACTICE:** Extend the Student class above with a method `letter_grade()` that returns 'A', 'B', 'C', or 'F' based on the average, using the grading logic from Week 2's IF lesson.

WEEK 4

Advanced Python & Pro Tools

Days 22–30: Work with modules, decorators, generators, regex, APIs, and databases — then build a full capstone project.

Why This Week Matters

The final week is less about new basic syntax and more about the tools that separate someone who knows Python syntax from someone who can build real, professional software. Modules let you reuse code across projects. Decorators and generators are patterns you'll recognize throughout the Python ecosystem. APIs and databases connect your programs to the outside world and to persistent data. This is also the least hand-held week — real programming problems never arrive with a lesson number attached to them.

This Week's Lessons

DAY 22	Modules & pip	Imports, packages, virtual envs
DAY 23	Decorators	Wrapping functions with extra behavior
DAY 24	Generators & Iterators	Lazy, memory-efficient sequences
DAY 25	Regular Expressions	Pattern matching in text
DAY 26	JSON & APIs	Talking to the web with requests
DAY 27	Databases with sqlite3	Persistent, queryable storage
DAY 28	Testing & Debugging	unittest and best practices
DAY 29	Clean Code & PEP 8	Writing code others can read
DAY 30	Project 2	Capstone: Expense Tracker app

By the End of This Week, You'll Be Able To

- Import and use both built-in and third-party (pip-installed) packages
- Write a decorator that adds behavior to an existing function
- Use regular expressions to find and validate patterns in text
- Fetch and parse data from a real web API
- Store and query data using a real SQLite database
- Build a complete, multi-feature capstone project from a set of requirements

📌 Teacher's Note

By this point, most of the remaining struggle isn't about understanding any single feature — it's about deciding which tool to reach for first when a real, messy problem shows up with no instructions attached. That's exactly what the Day 30 capstone project is designed to test, and it's the truest measure of whether the last four weeks actually stuck.

Modules, pip & Virtual Environments

📌 Teacher's Note

Modules are where students realize Python isn't just what's built in — the standard library and community packages (via pip) extend it enormously. I like to frame pip install as "borrowing someone else's already-solved problem" rather than a mysterious command to memorize. Virtual environments feel unnecessary to a beginner working on one small script, but they become obviously necessary the first time two projects need conflicting versions of the same package.

What Is a Module?

A **module** is simply a Python file full of reusable code — functions, classes, and variables — that you can import into another program instead of rewriting. Python ships with a huge built-in collection called the **standard library**.

Importing Built-in Modules

CODE

```
import math
import random

print(math.sqrt(16))
print(math.pi)
print(random.randint(1, 10))
```

OUTPUT

```
4.0
3.141592653589793
7
```

Different Ways to Import

CODE

```
import math
print(math.sqrt(25))

from math import sqrt
print(sqrt(25))

from math import sqrt as square_root
print(square_root(25))
```

OUTPUT

```
5.0
5.0
5.0
```

Style	When to use it
import module	Default — clear where each function comes from
from module import name	When you use one function often and want shorter code
import module as alias	When the module name is long (e.g. import pandas as pd)

Creating Your Own Module

Any .py file can be imported into another. If you save this as **helpers.py**:

CODE

```
# helpers.py
def greet(name):
    return f"Hello, {name}!"
```


You can then use it from another file in the same folder:

CODE

```
# main.py
import helpers
print(helpers.greet("Aisha"))
```

OUTPUT

```
Hello, Aisha!
```

Installing Third-Party Packages with pip

pip is Python's package installer — it downloads code other people have written and published, so you don't have to build everything from scratch.

TERMINAL

```
pip install requests
```

CODE

```
import requests
response = requests.get("https://api.github.com")
print(response.status_code)
```

OUTPUT

```
200
```

Virtual Environments — Why They Matter

A **virtual environment** is an isolated space for one project's packages, so different projects can use different (even conflicting) package versions without interfering with each other.

TERMINAL

```
python -m venv myenv
myenv\Scripts\activate      (Windows)
source myenv/bin/activate   (Mac/Linux)
```

💡 **TIP:** Get in the habit of creating a fresh virtual environment for every new project — it's a small extra step that prevents a lot of confusing 'it works on my machine' problems later.

⇒ **PRACTICE:** Install the requests package with pip. Write a script that imports it and fetches <https://api.github.com>, printing the response's status code and the first 100 characters of its text.

Quick Check

What's the difference between `import math` and `from math import sqrt`?

Answer: `import math` requires writing `math.sqrt()`; `from math import sqrt` lets you call `sqrt()` directly without the module prefix.

WEEK 4 — Pro Tools • DAY 23 OF 30

Decorators

👨‍🏫 Teacher's Note

Decorators are the concept most likely to make an intermediate student feel lost again, right after they'd started feeling confident. I always connect it back to Day 13's lambda lesson: Python functions are values, which means a function can take another function as input and return a new one. A decorator is just that idea applied to a common, useful pattern — wrapping extra behavior around an existing function without rewriting it.

Functions Are Values

In Python, a function is just another kind of value — it can be stored in a variable, passed into another function, or returned from one. This idea is the foundation decorators are built on.

CODE

```
def shout(text):  
    return text.upper()  
  
my_func = shout  
print(my_func("hello"))
```

OUTPUT

```
HELLO
```

A Function That Takes Another Function

CODE

```
def loud(func):  
    def wrapper(text):  
        return func(text) + "!!!"  
    return wrapper  
  
def greet(name):  
    return f"Hello, {name}"  
  
loud_greet = loud(greet)  
print(loud_greet("Aisha"))
```

OUTPUT

```
Hello, Aisha!!!
```

The @decorator Syntax

A **decorator** is exactly the pattern above, with special @ syntax that makes it cleaner to apply.

CODE

```
def loud(func):  
    def wrapper(text):  
        return func(text) + "!!!"  
    return wrapper  
  
@loud  
def greet(name):  
    return f"Hello, {name}"  
  
print(greet("Aisha"))
```

OUTPUT

```
Hello, Aisha!!!
```

@loud above **def greet(...)** is exactly equivalent to writing `greet = loud(greet)` — the decorator wraps extra behavior around the original function.

A Practical Example: Timing a Function

CODE

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end - start:.4f} seconds")
        return result
    return wrapper

@timer
def slow_square(n):
    time.sleep(1)
    return n * n

print(slow_square(5))
```

OUTPUT

```
slow_square took 1.0002 seconds
25
```

A Practical Example: Logging Calls

CODE

```
def log_call(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__} with {args}")
        return func(*args, **kwargs)
    return wrapper

@log_call
def add(a, b):
    return a + b

print(add(3, 4))
```

OUTPUT

```
Calling add with (3, 4)
7
```

💡 **TIP:** You don't need to master writing your own decorators immediately — recognizing @decorator syntax and understanding roughly what it does is the more important first step, since you'll see decorators constantly in real Python code (like @app.route in Flask).

⇒ **PRACTICE:** Write a decorator called shout_result that makes any function's return value uppercase before returning it. Apply it to a function that returns a greeting message.

Quick Check

What does *args and **kwargs inside a decorator's wrapper function allow?

Answer: It lets the decorator work on ANY function, regardless of how many positional or keyword arguments that function takes.

WEEK 4 — Pro Tools • DAY 24 OF 30

Generators & Iterators

👨‍🏫 Teacher's Note

Generators solve a problem students usually haven't hit yet — running out of memory processing something huge — so the motivation has to be explained before the syntax will make sense. I frame it as: a normal function hands you the whole answer at once; a generator hands you one answer at a time, only when you ask for it. That laziness is the entire value proposition.

The Problem Generators Solve

A normal function that builds a huge list uses a lot of memory — the entire list must exist at once. A **generator** produces values **one at a time, on demand**, without ever holding the whole sequence in memory at once.

A Normal Function vs a Generator

CODE

```
def get_squares(n):  
    result = []  
    for i in range(n):  
        result.append(i * i)  
    return result  
  
print(get_squares(5))
```

OUTPUT

```
[0, 1, 4, 9, 16]
```

CODE

```
def get_squares_gen(n):  
    for i in range(n):  
        yield i * i  
  
for square in get_squares_gen(5):  
    print(square)
```

OUTPUT

```
0  
1  
4  
9  
16
```

The keyword **yield** is what makes this a generator instead of a normal function — each time the loop asks for the next value, the function resumes exactly where it left off.

Generators Are Lazy — Values Are Made Only When Needed

CODE

```
def count_up_to(n):  
    i = 1  
    while i <= n:  
        yield i  
        i += 1  
  
counter = count_up_to(3)  
print(next(counter))  
print(next(counter))  
print(next(counter))
```

OUTPUT

```
1  
2  
3
```

Why This Matters for Big Data

CODE

```
def read_large_file(path):
    with open(path) as f:
        for line in f:
            yield line.strip()

# Only one line is in memory at a time, even for a huge file
# for line in read_large_file("huge_log.txt"):
#     print(line)
```

Generator Expressions — A Comprehension That's Lazy

Just like a list comprehension, but with parentheses instead of square brackets — it produces values one at a time instead of building the whole list immediately.

CODE

```
squares = (x * x for x in range(5))
print(squares)
print(list(squares))
```

OUTPUT

```
<generator object <genexpr> at 0x0000001>
[0, 1, 4, 9, 16]
```

List comprehension []	Generator expression ()
Builds the whole list immediately in memory	Produces one value at a time, on demand
Can be looped over multiple times	Can only be looped over ONCE
Better for small data you'll reuse	Better for large data or one-time processing

⚠ **WATCH OUT:** A generator can only be iterated through once — after it's exhausted, looping over it again produces nothing. If you need the values more than once, convert it to a list with `list(...)` first.

⇒ **PRACTICE:** Write a generator function called `even_numbers(limit)` that yields even numbers from 2 up to `limit`. Loop through it and print each value.

Quick Check

What keyword turns a regular function into a generator?

Answer: `yield`

WEEK 4 — Pro Tools • DAY 25 OF 30

Regular Expressions

👨 Teacher's Note

Regular expressions have a steep-looking learning curve, but the underlying idea is simple: describing a pattern of text instead of an exact piece of text. I never ask students to memorize the whole syntax table — I teach the five or six symbols that cover 90% of real use cases and point them to documentation for the rest. Trying to memorize every regex symbol on day one is a fast way to make students hate the topic.

What Is a Regular Expression?

A **regular expression** (regex) is a way of describing a **pattern** of text, rather than exact text — perfect for validating input (emails, phone numbers) or searching through large amounts of text.

CODE

```
import re

text = "My email is aisha@example.com"
match = re.search(r"\w+@\w+\.\w+", text)
print(match.group())
```

OUTPUT

aisha@example.com

Core Regex Symbols (the 90% you'll actually use)

Symbol	Meaning	Example matches
\d	A digit (0-9)	\d\d\d matches "123"
\w	A word character (letter, digit, underscore)	\w+ matches "hello_1"
\s	A whitespace character	\s matches a space or tab
+	One or more of the previous symbol	\d+ matches "42" or "42042"
*	Zero or more of the previous symbol	
?	Zero or one (optional)	colou?r matches color or colour
.	Any character except a newline	
^ / \$	Start / end of the string	

re.search() vs re.findall() vs re.match()

CODE

```
import re
text = "Call 9876543210 or 8765432109"

print(re.search(r"\d{10}", text).group())
print(re.findall(r"\d{10}", text))
```

OUTPUT

```
9876543210
['9876543210', '8765432109']
```

Function	Returns
re.search()	The FIRST match anywhere in the string, or None
re.match()	A match only at the START of the string, or None
re.findall()	A list of ALL matches in the string
re.sub()	The string with matches replaced by something else

Validating an Email Format

CODE

```
import re

def is_valid_email(email):
    pattern = r"^[\\w.]+@[\\w]+\\. [a-zA-Z]{2,}$"
    return re.match(pattern, email) is not None

print(is_valid_email("aisha@example.com"))
print(is_valid_email("not-an-email"))
```

OUTPUT

```
True
False
```

Replacing Text with re.sub()

CODE

```
import re
text = "Contact: 987-654-3210"
cleaned = re.sub(r"[-\s]", "", text)
print(cleaned)
```

OUTPUT

Contact:9876543210

🔗 **TIP:** Don't try to memorize the entire regex syntax table at once — learn the handful of symbols above deeply, and look up anything more exotic when you actually need it. Real developers reference regex docs constantly.

⇒ **PRACTICE:** Write a function that uses `re.findall()` to extract all phone numbers (format: 10 digits) from a block of text containing several sentences with numbers mixed in.

Quick Check

What does the regex pattern `\d+` match?

Answer: One or more consecutive digits, like "5", "42", or "1234"

WEEK 4 — Pro Tools • DAY 26 OF 30

Working with JSON & APIs

📌 Teacher's Note

Working with JSON and APIs is usually the first moment Python feels connected to the real, live internet rather than just the local machine, and that shift in perceived usefulness is worth highlighting explicitly. I always show what a raw API response looks like as text before parsing it into a Python dictionary — seeing the messy original makes the value of `json.loads()` obvious rather than assumed.

What Is JSON?

JSON (JavaScript Object Notation) is a text format for representing structured data — it looks almost identical to a Python dictionary, and it's the standard way data travels across the web via APIs.

CODE

```
{
  "name": "Aisha",
  "age": 21,
  "is_student": true,
  "courses": ["Python", "Math"]
}
```

Converting Between JSON and Python

CODE

```
import json

data = {"name": "Aisha", "age": 21}

json_text = json.dumps(data)
print(json_text)
print(type(json_text))

back_to_dict = json.loads(json_text)
print(back_to_dict["name"])
```

OUTPUT

```
{"name": "Aisha", "age": 21}
<class 'str'>
Aisha
```

Function	Direction
<code>json.dumps(obj)</code>	Python object → JSON text (for sending/saving)
<code>json.loads(text)</code>	JSON text → Python object (for using the data)
<code>json.dump(obj, file)</code>	Python object → written directly to a file
<code>json.load(file)</code>	JSON file → Python object

What Is an API?

An **API** (Application Programming Interface) lets your program request data from another service over the internet — weather data, currency rates, GitHub info, and thousands of other sources.

Making a Request with the requests Library

CODE

```
import requests

response = requests.get("https://api.github.com/users/python")
data = response.json()

print(response.status_code)
print(data["name"])
print(data["public_repos"])
```

OUTPUT

```
200
Python
4
```

Status code	Meaning
200	Success
404	Not found
401	Unauthorized (missing/invalid API key)
500	Server error (not your fault)

Handling a Failed Request Safely

CODE

```
import requests

try:
    response = requests.get("https://api.github.com/users/python", timeout=5)
    response.raise_for_status()
    data = response.json()
    print(data["name"])
except requests.exceptions.RequestException as e:
    print(f"Request failed: {e}")
```

OUTPUT

```
Python
```

⚠ **WATCH OUT:** Never assume an API call will succeed — networks fail, servers go down, and rate limits get hit. Always wrap real API calls in try/except, and check `response.status_code` before trusting the data.

➡ **PRACTICE:** Use requests to call `https://api.github.com/users/` for a GitHub username of your choice, and print their name, number of public repos, and account creation date from the returned JSON.

Quick Check

Which function converts a Python dictionary into a JSON-formatted string?

Answer: `json.dumps()`

WEEK 4 — Pro Tools • DAY 27 OF 30

Working with Databases: sqlite3

📌 Teacher's Note

Databases are often postponed until 'advanced' courses, but a basic sqlite3 lesson early on pays off — it reframes data as something that outlives a single program run, building on the file-handling lesson from Week 3. I keep this lesson intentionally narrow: create a table, insert data, query it back. The goal is comfort with the pattern, not database design theory.

Why Use a Database Instead of a File?

You've already saved data to text files (Day 17) — a database is a more powerful upgrade: it lets you store much larger amounts of structured data and **query** it (search, filter, sort) efficiently, without manually parsing text.

sqlite3 — A Database Built Into Python

SQLite stores an entire database in a single file, and Python includes support for it with no extra installation — perfect for learning and for small-to-medium applications.

Connecting and Creating a Table

CODE

```
import sqlite3

conn = sqlite3.connect("students.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY,
    name TEXT,
    score INTEGER
)
""")
conn.commit()
```

Inserting Data

CODE

```
cursor.execute("INSERT INTO students (name, score) VALUES (?, ?)", ("Aisha", 92))
cursor.execute("INSERT INTO students (name, score) VALUES (?, ?)", ("Ravi", 78))
conn.commit()
```

⚠ **WATCH OUT:** Always use ? placeholders and pass values as a tuple, as shown above, instead of inserting variables directly into the SQL string with an f-string. Building queries with an f-string opens the door to SQL injection — a serious security risk.

Querying Data

CODE

```
cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()
for row in rows:
    print(row)
```

OUTPUT

```
(1, 'Aisha', 92)
(2, 'Ravi', 78)
```

Filtering with WHERE

CODE

```
cursor.execute("SELECT name FROM students WHERE score >= 80")
top_students = cursor.fetchall()
print(top_students)
```

OUTPUT

```
[('Aisha',)]
```

Updating and Deleting

CODE

```
cursor.execute("UPDATE students SET score = ? WHERE name = ?", (95, "Ravi"))
cursor.execute("DELETE FROM students WHERE name = ?", ("Aisha",))
conn.commit()
```

Always Close the Connection

CODE

```
conn.close()
```

The Basic Workflow, Summarized

Step	Command
Connect	sqlite3.connect("file.db")
Get a cursor	conn.cursor()
Run SQL	cursor.execute(sql, params)
Save changes	conn.commit()
Read results	cursor.fetchall() or .fetchone()
Clean up	conn.close()

⇒ **PRACTICE:** Create a SQLite database called library.db with a table 'books' (id, title, author, is_borrowed). Insert 3 books, then write a query that selects only books where is_borrowed = 0.

Quick Check

Why should you use ? placeholders instead of f-strings when inserting values into SQL?

Answer: To prevent SQL injection — a security vulnerability where malicious input could manipulate or damage the database.

WEEK 4 — Pro Tools • DAY 28 OF 30

Testing & Debugging: unittest

📌 Teacher's Note

Testing is the topic students most often skip when self-teaching, and it shows later in messier, harder-to-debug code. I frame writing a test not as extra work but as writing down what you already believe should be true about your function — if that belief turns out to be wrong, you want to find out in ten seconds from a test, not twenty minutes into debugging a much larger program.

Why Write Tests?

A **test** is a small piece of code that checks whether another piece of code behaves the way you expect. Instead of manually re-checking your program every time you change something, a test suite checks it automatically, in seconds.

A Function to Test

CODE

```
def add(a, b):
    return a + b
```

Writing a Test with unittest

CODE

```
import unittest

def add(a, b):
    return a + b

class TestAddFunction(unittest.TestCase):
    def test_positive_numbers(self):
        self.assertEqual(add(2, 3), 5)

    def test_negative_numbers(self):
        self.assertEqual(add(-1, -1), -2)

    def test_zero(self):
        self.assertEqual(add(0, 5), 5)

unittest.main(argv=[""], exit=False)
```

OUTPUT

```
..
-----
Ran 3 tests in 0.001s

OK
```

Common Assertion Methods

Method	Checks that...
<code>assertEqual(a, b)</code>	<code>a == b</code>
<code>assertNotEqual(a, b)</code>	<code>a != b</code>
<code>assertTrue(x)</code>	<code>x</code> is True (or truthy)
<code>assertFalse(x)</code>	<code>x</code> is False (or falsy)
<code>assertRaises(Error, func, args)</code>	Calling <code>func(args)</code> raises the given Error

Testing That an Error Is Raised

CODE

```
import unittest

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b

class TestDivide(unittest.TestCase):
    def test_divide_by_zero_raises(self):
        with self.assertRaises(ValueError):
            divide(10, 0)

unittest.main(argv=[""], exit=False)
```

OUTPUT

```
..
-----
Ran 1 test in 0.000s

OK
```

Debugging Without a Formal Test Suite

Technique	When to use it
print() statements	Quick checks of a variable's value at a specific point
Reading the error traceback bottom-up	The last line usually shows the actual error type and message
Isolating the problem	Comment out code until the error disappears, then add it back piece by piece
Using a debugger (VS Code has one built in)	Step through code line by line, inspecting variables as you go

🔗 **TIP:** A test doesn't need to be complicated to be valuable. Even one or two tests for your most important functions catch a surprising number of bugs before they reach real users.

⇒ **PRACTICE:** Write a function `is_palindrome(text)` that returns `True` if a string reads the same forwards and backwards. Write 3 unittest test cases for it: a palindrome, a non-palindrome, and an empty string.

Quick Check

What does `self.assertRaises(ValueError)` check for?

Answer: That the code inside the 'with' block actually raises a `ValueError` — the test fails if no error (or the wrong type of error) occurs.

WEEK 4 — Pro Tools • DAY 29 OF 30

Clean Code & PEP 8 Best Practices

📌 Teacher's Note

By Day 29, the goal shifts from learning new syntax to writing code other people (and future-you) can actually read. PEP 8 conventions feel like arbitrary rules until a student has to read someone else's inconsistently-styled code — then consistent style stops feeling optional. I have students trade code with a partner for five minutes purely to identify what was hard to read, no fixing allowed, just noticing.

Why Code Style Matters

Code is read far more often than it's written — by teammates, by future employers reviewing a portfolio, and by your own future self months later. **PEP 8** is Python's official style guide, and following it makes your code instantly more professional and easier for others to read.

Key PEP 8 Conventions

Rule	Example
Use snake_case for variables and functions	<code>total_score</code> , not <code>TotalScore</code>
Use PascalCase for class names	<code>BankAccount</code> , not <code>bank_account</code>
Use UPPER_CASE for constants	<code>MAX_ATTEMPTS = 3</code>
4 spaces per indentation level	never tabs mixed with spaces
2 blank lines between top-level functions/classes	
Keep lines under ~79-99 characters	
Add a space after commas	<code>func(a, b, c)</code> not <code>func(a,b,c)</code>

Before and After: A Readability Comparison

CODE

```
def calc(a,b,t):
    if t=="add":
        return a+b
    elif t=="sub":
        return a-b
```

CODE

```
def calculate(num1, num2, operation):
    if operation == "add":
        return num1 + num2
    elif operation == "subtract":
        return num1 - num2
```

Both versions run identically — but the second is dramatically easier for another person (or future you) to understand at a glance.

Writing Good Comments and Docstrings

CODE

```
def calculate_discount(price, percent):
    """Return the discounted price given an original price and a discount percent."""
    return price - (price * percent / 100)
```

A **docstring** (the triple-quoted string right after a function definition) documents what the function does — many tools and editors display it automatically when someone uses your function.

Meaningful Naming

Poor naming	Better naming
x, y, temp, data	price, tax_rate, user_input
def f(a, b):	def calculate_total(price, tax):
lst, lst2	students, top_students

DRY — Don't Repeat Yourself

If you find yourself copy-pasting the same few lines with tiny changes, that's usually a sign the logic belongs in a function instead. This single principle prevents an enormous number of bugs, since a fix only ever needs to happen in one place.

💡 **TIP:** Most professional teams use an automatic formatter (like black) and a linter (like flake8 or pylint) so style consistency isn't something anyone has to remember manually — worth exploring once you're comfortable with the basics.

⇒ **PRACTICE:** Take a messy function you wrote earlier in this course and rewrite it following PEP 8: proper naming, spacing, and a docstring explaining what it does.

Quick Check

What naming convention does PEP 8 recommend for Python variables and functions?

Answer: *snake_case* — lowercase words separated by underscores, like *total_score*

FINAL PROJECT • BEGINNER TO PRO

Project 2 (Capstone): Expense Tracker

Project Overview

Your capstone is a **Personal Expense Tracker** — a command-line application that lets a user log expenses, view spending by category, and persist all data in a real SQLite database. This project draws on skills from **every single week** of this course: functions, OOP, file/database handling, error handling, and clean code practices.

Requirements

- ☐ Store each expense with: description, amount, category, and date
- ☐ Save all expenses permanently using **sqlite3** (not just in memory)
- ☐ Let the user add a new expense
- ☐ Let the user view all expenses
- ☐ Let the user view total spending by category
- ☐ Handle invalid input gracefully (e.g. entering text where a number is expected)
- ☐ Organize the code using a class, not just loose functions

Step 1: The ExpenseTracker Class

CODE

```
import sqlite3
from datetime import date

class ExpenseTracker:
    def __init__(self, db_name="expenses.db"):
        self.conn = sqlite3.connect(db_name)
        self.cursor = self.conn.cursor()
        self.cursor.execute("""
            CREATE TABLE IF NOT EXISTS expenses (
                id INTEGER PRIMARY KEY,
                description TEXT,
                amount REAL,
                category TEXT,
                date TEXT
            )
        """)
        self.conn.commit()

    def add_expense(self, description, amount, category):
        today = str(date.today())
        self.cursor.execute(
            "INSERT INTO expenses (description, amount, category, date) VALUES (?, ?, ?, ?)",
            (description, amount, category, today)
        )
        self.conn.commit()
        print(f"Added: {description} - {amount} ({category})")

    def view_all(self):
        self.cursor.execute("SELECT * FROM expenses ORDER BY date DESC")
        rows = self.cursor.fetchall()
        if not rows:
            print("No expenses recorded yet.")
            return
        for row in rows:
            _id, desc, amount, category, expense_date = row
            print(f"{expense_date} | {category:12} | {desc:20} | {amount:.2f}")

    def total_by_category(self):
        self.cursor.execute(
            "SELECT category, SUM(amount) FROM expenses GROUP BY category"
        )
        results = self.cursor.fetchall()
        for category, total in results:
            print(f"{category}: {total:.2f}")

    def close(self):
        self.conn.close()
```

Step 2: The Menu Loop with Error Handling

CODE

```
def main():
    tracker = ExpenseTracker()

    while True:
        print("\n--- Expense Tracker ---")
        print("1. Add expense")
        print("2. View all expenses")
        print("3. Total by category")
        print("4. Quit")
        choice = input("Choose an option: ")

        if choice == "1":
            description = input("Description: ")
            category = input("Category (e.g. Food, Rent, Travel): ")
            try:
                amount = float(input("Amount: "))
                tracker.add_expense(description, amount, category)
            except ValueError:
                print("Invalid amount — please enter a number.")
        elif choice == "2":
            tracker.view_all()
        elif choice == "3":
            tracker.total_by_category()
        elif choice == "4":
            tracker.close()
            print("Goodbye!")
            break
        else:
            print("Invalid choice, try again.")

main()
```

Sample Run

TERMINAL

```
--- Expense Tracker ---
1. Add expense
2. View all expenses
3. Total by category
4. Quit
Choose an option: 1
Description: Groceries
Category (e.g. Food, Rent, Travel): Food
Amount: 45.50
Added: Groceries - 45.5 (Food)

--- Expense Tracker ---
Choose an option: 3
Food: 45.50

--- Expense Tracker ---
Choose an option: 4
Goodbye!
```

Self-Assessment Rubric (for teaching use)

Criteria	Points
Data is correctly saved to and read from SQLite	25
All 4 menu options work as expected	25
Invalid input is handled without crashing the program	20
Code uses a class with clear, well-named methods	20
Bonus: category filtering, date-range search, or CSV export	10

Extend the Project (Recommended)

- ☐ Add a "delete expense by ID" option
- ☐ Let the user filter expenses by a specific month
- ☐ Export all expenses to a CSV file using what you learned on Day 17
- ☐ Add a monthly budget per category and warn the user when they go over

🔗 **TIP:** This capstone is intentionally open-ended past the core requirements. If you're teaching this course, treat the extensions as the real differentiator between students — anyone can follow the steps above, but choosing and implementing a good extension shows real understanding.

30-Day Journey Recap

Week	Focus
Week 1	Variables, data types, and the four core data structures
Week 2	Conditionals, loops, functions — and Project 1: Contact Book
Week 3	Comprehensions, files, exceptions, and OOP
Week 4	Modules, decorators, generators, regex, APIs, databases — and Project 2: Expense Tracker

Where to Go Next

- Explore a web framework like **Flask** or **Django** to build full websites
- Learn **pandas** and **NumPy** if you're interested in data analysis
- Practice on real coding-challenge sites to sharpen problem-solving speed
- Read other people's open-source Python code — it teaches idioms no course can fully cover
- Teach someone else — explaining a concept is the fastest way to master it

⇒ **PRACTICE:** Build the complete Expense Tracker as shown, test all 4 menu options, then implement at least one extension from the list above on your own.